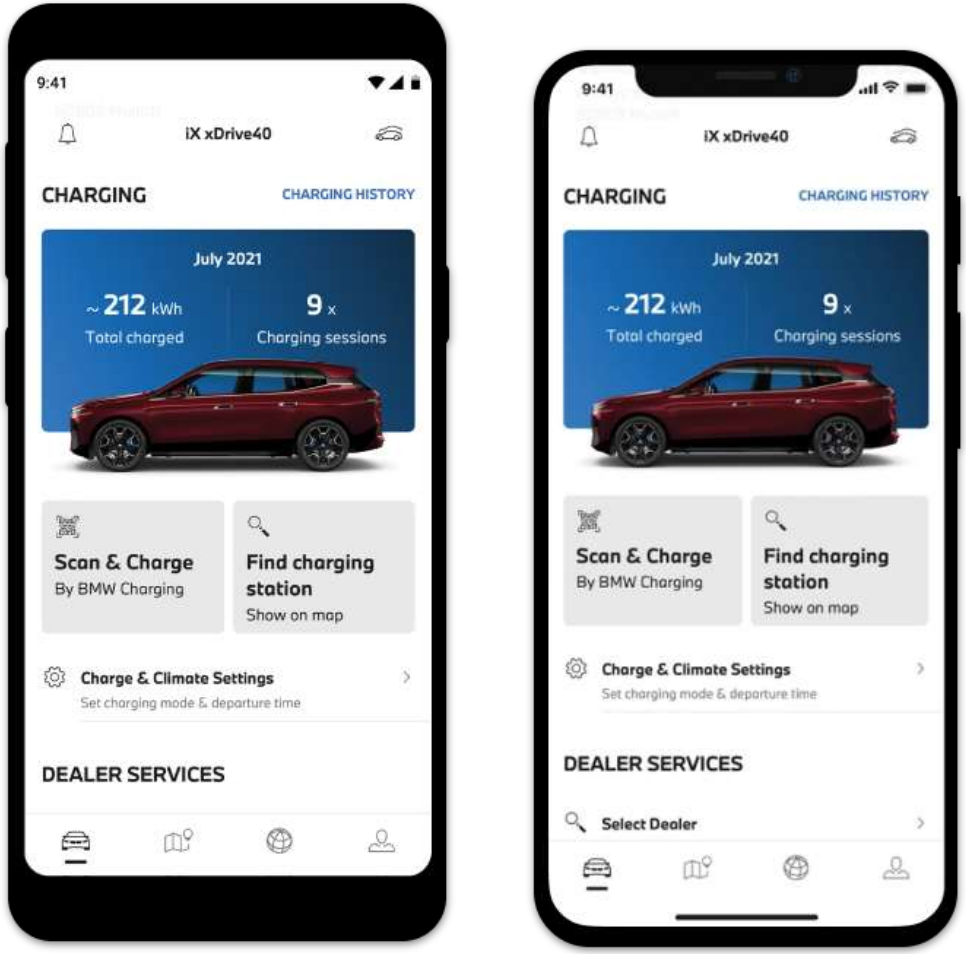
Introduction

Mobile Application Development

Training Objective

Understand principles and best practices of **cross-platform mobile application development** using the Flutter framework



Learning outcomes

Flutter

After Completing this training you should be able to

- explain the **fundamentals** of the **Flutter framework**
- create **UIs** using **Flutter Widgets**
- explore Flutter Widget Libraries

  Layout Widgets, **Scrollable Widgets** and **Interactive Widgets**
- **navigate** between screens

Learning outcomes

Flutter

After Completing this training you should be able to

- make **network requests**
- parse **JSON** objects
- manage **application states**
- use state management libraries such as **Bloc, Riverpod**

Learning outcomes

Flutter

After Completing this training you should be able to

- explore different software **application architectures**
- explain the steps and processes involved to **release** your apps to the iOS App Store and Google Play Store

https://flutter.dev/

Native vs Cross Platform

Native apps are exclusively developed for a single platform, such as

- `iOS`, `Android`, or `Windows`

The Native apps have direct access to the hardware of the device such as

- a microphone, camera, GPS, and many more
- they acquire all the possible advantages of the device and deliver a high-performance and better user experience

Native vs Cross Platform

Cross-platform app development is the process to develop an app that is compatible with various platforms such as `iOS` or `Android`

Cross platform development frameworks

- `Flutter` (https://github.com/flutter/flutter),
- `React Native` (https://github.com/facebook/react-native)

Cross Platform Mobile App Development Options

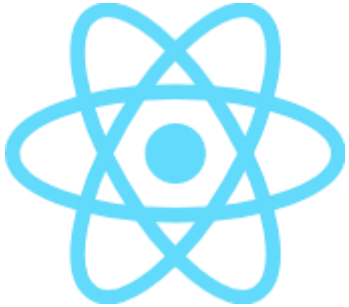
React Native (`https://reactnative.dev`)

Create native apps for Android and iOS using React

Written in JavaScript - rendered with native code

Native Development

Fast Refresh - See your changes as soon as you save



Cross Platform Mobile App Development Options

Flutter (https://flutter.dev/)

Flutter is an open source framework by Google for building beautiful, natively compiled, multi-platform applications from a single codebase

Powered by **Dart** Language

Flutter

Flutter Great Features

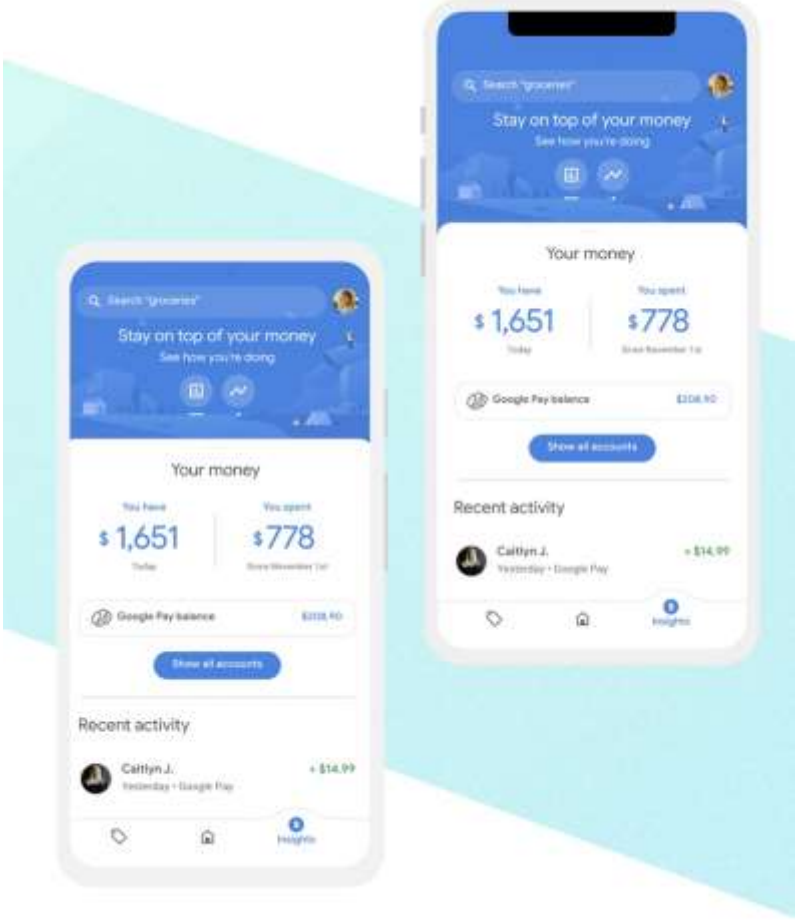
It is **open-source**

https://github.com/flutter/flutter

Flutter Great Features

It uses the **Dart** programming language

Dart is a modern, **UI-focused language** that's **compiled to native ARM** or **x86 code** or **cross-compiled to Javascript**



Flutter Great Features

It supports **hot reload**

> Hot reload allows you to **make updates to the code and the UI** that rebuild the widget tree, then deliver them live to emulators and devices — **without having to reload state or recompile your app**
>
> hot reload helps you quickly and easily experiment, build UIs, add features, and fix bugs faster

Flutter Great Features

It supports **hot restart**

Hot restart takes a little longer than hot reload because it loads the changes, restarts the app and resets the state

You need to use a full restart when you make certain significant changes to the code, including anything changing state management

Flutter Great Features

It supports **Material Design** and **Cupertino**

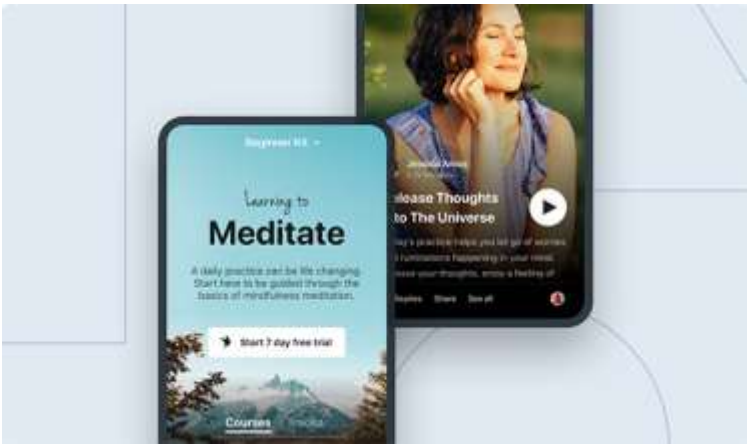
Material Design is a **design system created by Google** to help teams build high-quality digital experiences for Android, iOS, Flutter, and the web

Cupertino implements the **current iOS design language** based on **Apple's Human Interface Guidelines**

Flutter Great Features

It comes with **great animations and transitions**,

Because **widgets are composable**, you can be creative and flexible with the UI



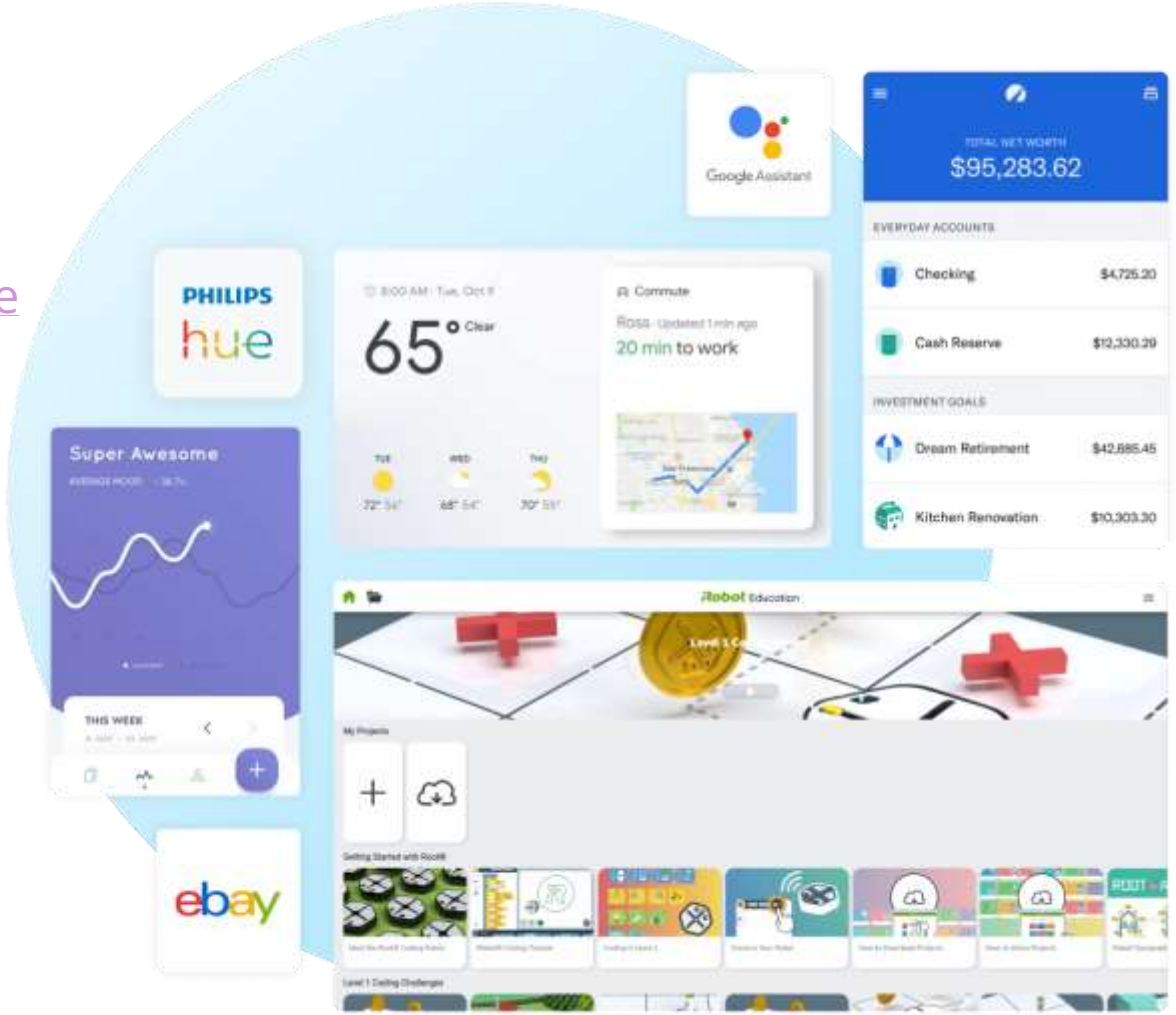
Flutter Great Features

It was designed with **accessibility** in mind, with out-of-the-box support for **dynamic font sizes** and **screen readers** and a ton of best practices around **language**, **contrast** and interaction methods

Flutter supports **C and C++ interoperability** as well as **platform channels** for connecting to Kotlin and Java on Android and Swift or Objective-C on iOS

Examples

https://flutter.dev/showcase



When not to use Flutter

Games and audio

for **complex 2D and 3D games**, you'd probably prefer to base your app on a cross-platform game engine technology like **Unity** or **Unreal**

cross-platform game engines have more domain-specific features like physics, sprite and asset management, game state management, multiplayer support and so on

flutter doesn't have a sophisticated audio engine yet, so audio editing or mixing apps are at a disadvantage over those that are purpose-built for a specific platform

When not to use Flutter

Apps with specific native SDK needs

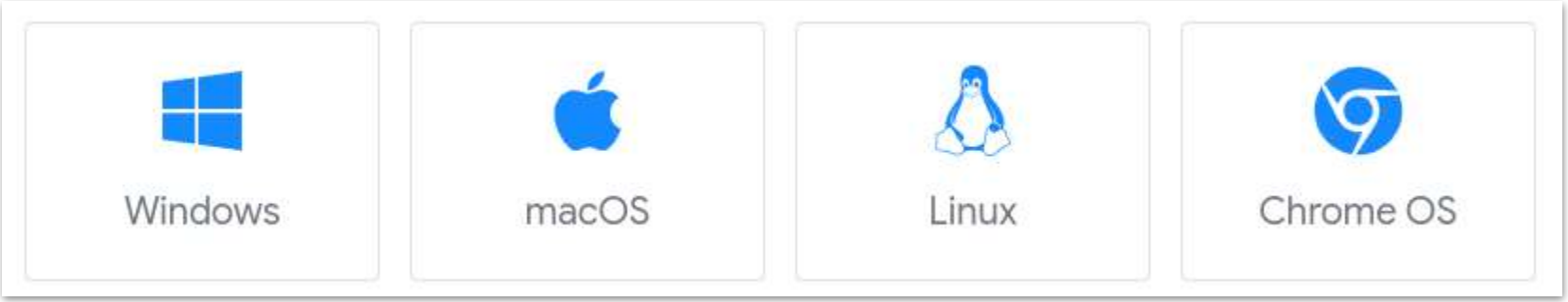
Flutter supports **many, but not all, native features**

if the app is highly integrated with a device's features and platform SDKs, it might be worth writing the app using the platform-specific tools.

Flutter also produces app binaries that are bigger in size than those built with platform frameworks

How to set up development platform

Go to `https://docs.flutter.dev/get-started/install` and follow the instruction for your platform



Install Android Studio

Flutter relies on a full installation of Android Studio to supply its Android platform dependencies

Go to https://developer.android.com/studio , download and install android studio

Set up an editor

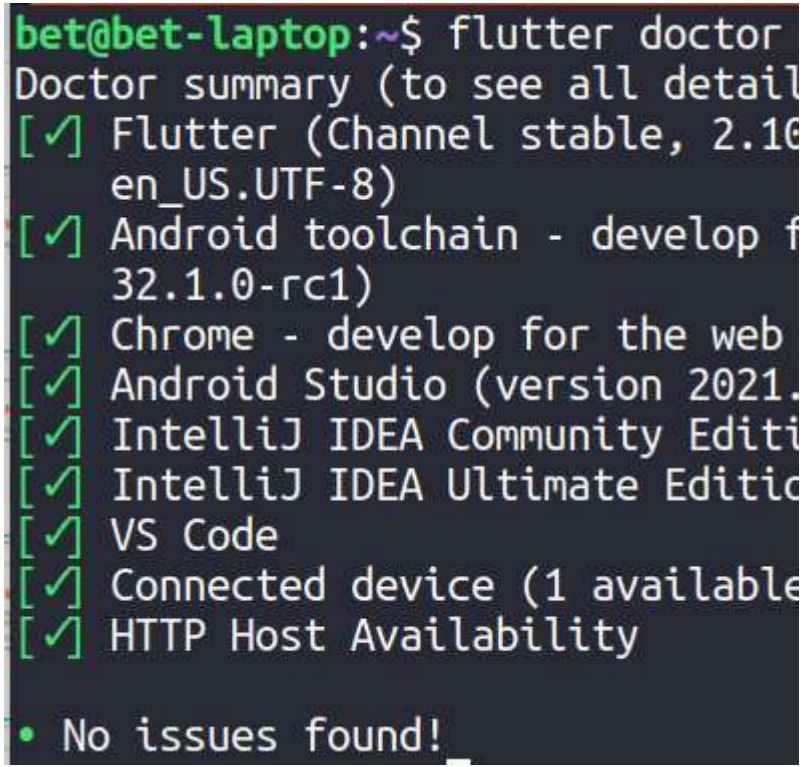
You can use **Android Studio** or **VS Code** to develop Flutter application

`https://docs.flutter.dev/get-started/editor`

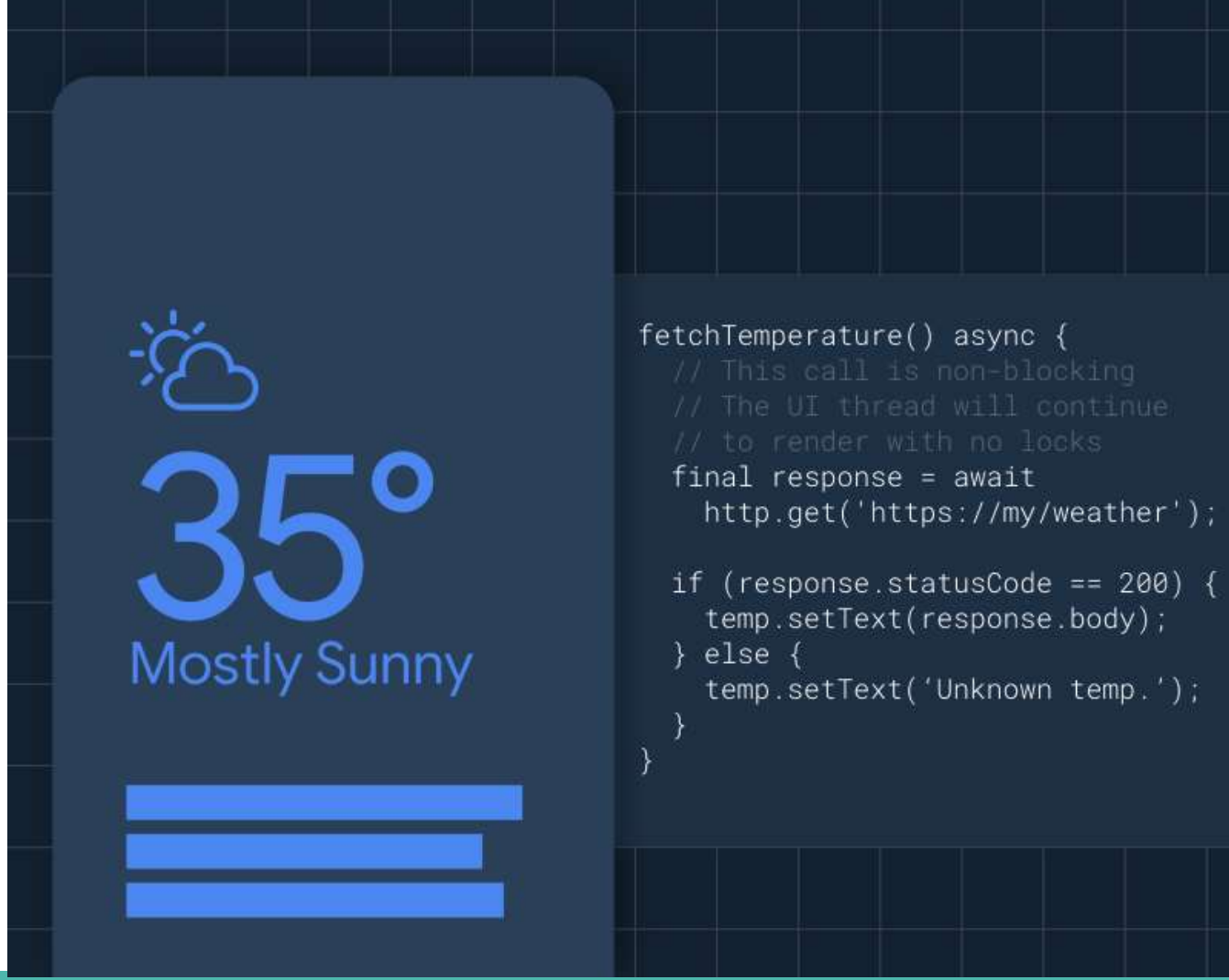
Set up validation

Validate your setup with the

`flutter doctor` command



Dart


35°
Mostly Sunny
fetchTemperature() async {
// This call is non-blocking
// The UI thread will continue
// to render with no locks
final response = await
http.get('https://my/weather');
if (response.statusCode == 200) {
temp.setText(response.body);
} else {
temp.setText('Unknown temp.');
}
}

What is Dart

Dart is a **client-optimized language** for fast apps on any platform

Dart provides the **language** and **runtimes** that **power Flutter apps**

https://dart.dev/overview

https://dart.dev/guides

Features of Dart

Optimized for UI

A programming language specialized around the needs of user interface creation

Productive development

Make changes iteratively: use hot reload to see the result instantly in your running app

`https://dart.dev/`

Features of Dart

Fast on all platforms

Compile to **ARM** & **x64 machine code** for mobile, desktop, and backend or

Compile to **JavaScript** for the web

`https://dart.dev/`

Dart: The language

`Type safe`

The Dart language is **type safe**

It uses static type checking to ensure that a variable's value always matches the variable's static type

Although types are mandatory, **type annotations are optional** because of **type inference**

Dart: The language

Null safety

Dart offers sound **null safety**, meaning that values can't be null unless you say they can be

```
int a = null; // Invalid in null-safe Dart

int? a = null; // Valid in null-safe Dart
```

Dart: The libraries

`dart:core`

Built-in types, collections, and other core functionality for every Dart program

`dart:collection`

Richer collection types such as queues, linked lists, hashmaps, and binary trees

Dart: The libraries

`dart:convert`

Encoders and decoders for converting between different data representations, including JSON and UTF-8

`dart:math`

Mathematical constants and functions, and random number generation

`dart:io`

File, socket, HTTP, and other I/O support for non-web applications

Dart: The libraries

`dart:async`

Support for asynchronous programming, with classes such as **`Future`** and **`Stream`**

`dart:isolate`

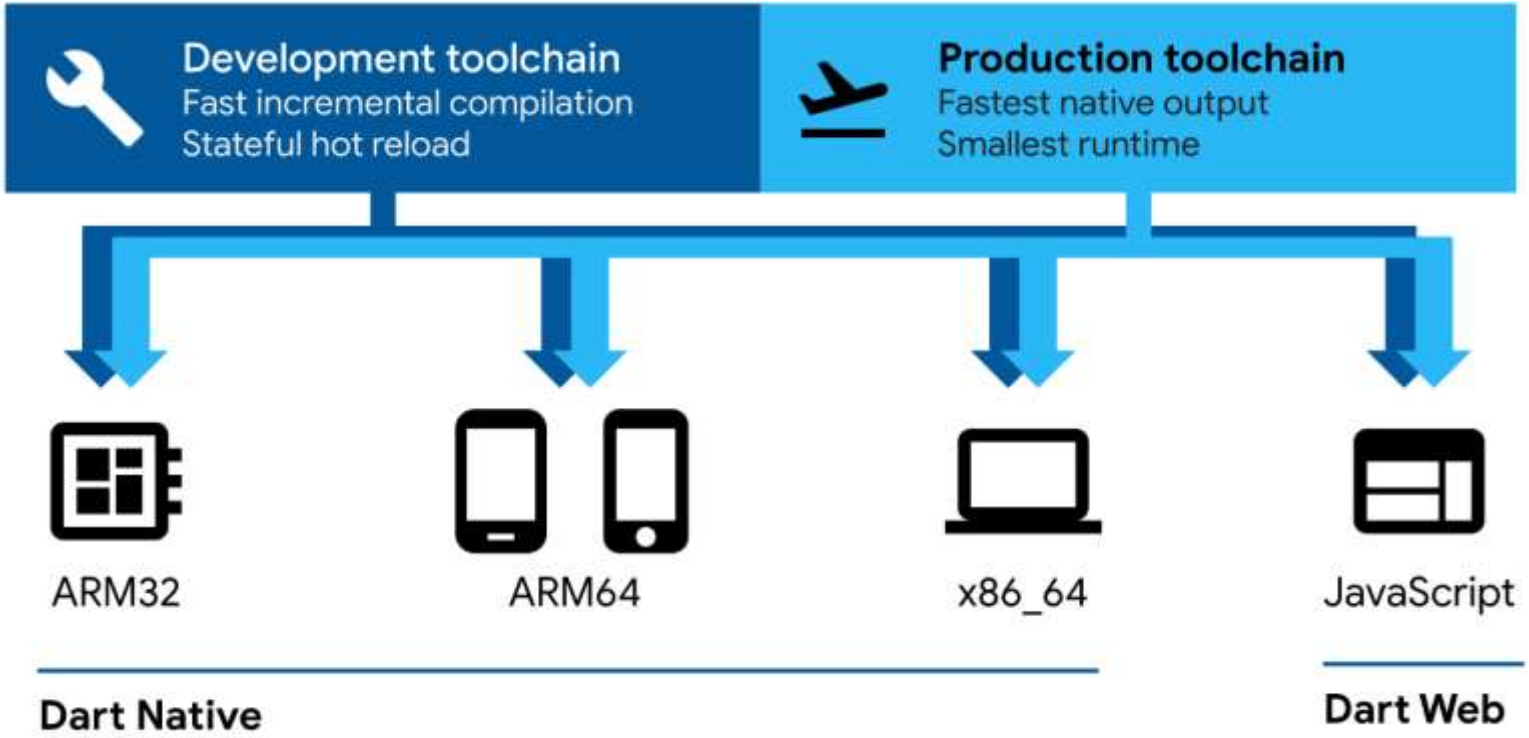
Concurrent programming using **`isolates`** — independent workers that are similar to threads but don't share memory, communicating only through messages

Dart: The libraries

`dart:html`

HTML elements and other resources for web-based applications that need to interact with the browser

Dart: The platforms



Dart: The platforms

Native platform: For apps targeting mobile and desktop devices, Dart includes both a Dart VM with **just-in-time (JIT)** compilation and an **ahead-of-time (AOT)** compiler for producing machine code

Web platform: For apps targeting the web, Dart includes both a **development time compiler (dartdevc)** and a **production time compiler (dart2js)**. Both compilers translate Dart into JavaScript

Brief introduction to Dart

Dart language features

Variables, Control flow statements, Functions, Comments, Async, Imports, Classes, Inheritance, Mixins, Interfaces and Abstract Classes, Exceptions

https://dart.dev/samples

Hello World

Every app has a `main()` function

To display text on the console, you can use the top-level `print()` function

```
void main() {
 print('Hello world');
}
```

Variables

Dart is **type-safe.** However, **most variables don’t need explicit types** because of dart`s **type inference** feature

```
1 var name = 'Voyager I';
2 var year = 1977;
3 var antennaDiameter = 3.7;
4 var flybyObjects = ['Jupiter', 'Saturn', 'Uranus'];
5 var image = {
    'tags': ['saturn'],
    'url': '//path/to/saturn.jpg'
  };
```

Guess the data type of the variable

Control flow statements: If Statements

```
var year = 2019;
if (year >= 2001) {
  print('21st century');
} else if (year >= 1901) {
  print('20th century');
}
```

Control flow statements: Loops

```
var shapes = ['Circle', 'Rectangle', 'Triangle'];

for(var shape in shapes){
  print(shape);
}
```

```
for (int i = 0; i < shapes.length; i++) {
  print(shapes[i]);
}
```

Control flow statements: Loops

```
var shapes = ['Circle', 'Rectangle', 'Triangle'];
int count = 0;
while (count < shapes.length) {
  print(shapes[count]);
  count++;
}
```

Functions

You can skip type of function parameters and return types but the best practice is to specify them

```
void main() {
 print(add(2,3));
}

add(x, y) {
 return x+y;
}
```

Recommended way of defining functions

```
int add(int x, int y) {
 return x+y;
}
```

Functions

A shorthand **=> (arrow)** syntax is handy **for functions that contain a single statement**

This syntax is especially useful when passing **anonymous functions** as arguments

```
void main() {
 print(add(2,3));
}
int add(int x, int y) => x+y;
```

Functions

A shorthand **=> (arrow)** syntax is is especially useful when passing **anonymous functions** as arguments

```
flyByObjects
    .where((name) => name.contains('turn'))
    .forEach(print);
```

Exercises

Write a dart function that find maximum between two numbers

Write a dart function that find the sum of all odd numbers between start and end integer values

Comments

```
// This is a normal, one-line comment.


/// This is a documentation comment, used to document

/// libraries, classes, and their members. Tools like IDEs
/// and dartdoc treat doc comments specially.


/* Multi-line comments like this is also

supported */
```

Imports

```
// Importing core libraries
import 'dart:math';


// Importing libraries from external packages
import 'package:test/test.dart';


// Importing files
import 'path/to/my_other_file.dart';
```

Class

```
import 'dart:math';
class Circle {
 double _radius;
 static const double PI = 3.14;
 Circle(this._radius);
 double area() => PI * pow(_radius, 2);
 get radius => _radius;
}
```

Notice the underscore (_)

How many **constructors**, **properties**, and **methods** are there?

What is the role of the _ symbol

Class

```dart
import 'dart:math';

class Circle {
 double _radius;
 static const double PI = 3.14;
 Circle(this._radius);
 double area() => PI * pow(_radius, 2);
 get radius => _radius;
}
```

Class with two properties, one constructor, one method and one getter method

Constructor

Getter

Inheritance

Dart has single inheritance

```dart
abstract class Shape {
 double area();
}
```

```dart
class Circle extends Shape {
 double _radius;
 static const double PI = 3.14;

 Circle(this._radius);

 @override
 double area() => PI * pow(_radius, 2);
 get radius => _radius;
}
```

Mixins

Mixins are a way of reusing code in multiple class hierarchies

```
mixin Status {
 bool fullTime;
 bool manager;
}
```

```
class Employee with Status{
 String name;
 String address;
 String salary;
}
```

Interfaces and abstract classes

Dart has no interface keyword

All classes implicitly define an interface

Therefore, you can implement any class

```
abstract class Shape {
 double area();
}
```

```
class Circle implements Shape {
 double _r;
 static const double PI = 3.14;
 Circle(this._r);
 double area() => PI * _r * _r;
}
```

Async

Avoid callback hell and make your code much more readable by using **async** and **await**.

```
const oneSecond = Duration(seconds: 1);

Future<void> printWithDelay(String message) async {

  await Future.delayed(oneSecond);

  print(message);

}
```

Exceptions

To raise an exception, use throw

```
void main(){
 int x, y;

 if(x == null || y == null) {
   throw StateError("x or y are null");
 }
}
```

Exceptions

To catch an exception, use a try statement with **on** or **catch** (or both)

```
void main() {
 int x = 1, y = 0;
 try {
   x / y;
 } catch (IntegerDivisionByZeroException) {
   print('Y cannot be zero');
 }
}
```

Resources

Dart

A tour of the Dart language (https://dart.dev/guides/language/language-tour)

Dart cheatsheet (https://dart.dev/codelabs/dart-cheatsheet)

Language Samples (https://dart.dev/samples)

Resources

Flutter

Flutter Apprentice, By Michael Katz, Kevin David Moore, Vincent Ngo & Vincenzo Guzzi 2021, 2nd Edition

Flutter Docs (https://flutter.dev/docs)

Flutter in Action, By Eric Windmill, 2020

Beginning App Development with Flutter: Create Cross-Platform Mobile Apps, By Rap Payne, 2019

Flutter Succinctly By Ed Freitas, 2019